

MyFood

Projeto da disciplina Programação 2 (COMP372) 2026.1 - UFAL desenvolvido em Java

Estrutura

```
MyFood/
├── lib/
│   └── easyaccept.jar          - EasyAccept
├── tests/                      - Testes do EasyAccept
└── src/br/ufal/ic/myfood/
    ├── Main.java              - Ponto de entrada
    ├── Facade.java            - Interface do Sistema
    ├── models/                - Entidades
    ├── managers/              - Regras de Negócio
    ├── utils/
    │   └── Persistencia.java   - persistência XML
    └── exceptions/            - Exceções
```

O MyFood é separado em camadas:

Camada de Interface: `br.ufal.ic.myfood.Facade`. Expõe os métodos para uso do EasyAccept, apenas delegando para o local correto.

Camada de Regras de Negócio: `br.ufal.ic.myfood.managers`. Responsáveis por coordenar as operações.

Camada de Modelo: `br.ufal.ic.myfood.models`. Representam as entidades do sistema (Usuário, Dono, Cliente, Empresa, Produto, Pedido).

Camada de Persistência: `br.ufal.ic.myfood.utils.Persistencia`. Implementa salvamento/carregamento em XML.

O ponto de entrada (Main) executa o EasyAccept, apontando para a Facade que delega o que se deve ser feito aos Managers.

Principais componentes e a suas interações

Main

Roda os testes do EasyAccept e define a Facade como interface do sistema.

Facade

A Facade instancia os Managers: `UsuarioManager`, `EmpresaManager`, `ProdutoManager`, `PedidoManager` e `EntregaManager`, e depois recebe os comandos e envia esses comandos ao Manager correto.

A Facade é responsável pelo ciclo de vida do sistema, a partir dos métodos:

- **encerrarSistema** - Salva os dados e encerra o sistema.
- **zerarSistema** - Apaga os dados salvos em XML e reinicia os Managers.

Managers

- **UsuarioManager**
 - Criação e consulta de atributos de usuários, Login, validações
 - Persistência em `data/usuarios.xml` (Usando `utils/Persistencia.java`)
 - Usuários podem ser Cliente, Dono ou Entregador
- **EmpresaManager**
 - Criação e consulta de atributos de empresas, validações
 - Persistência em `data/empresas.xml` (Usando `utils/Persistencia.java`)
 - Depende de `UsuarioManager`
 - Empresas podem ser Restaurante, Mercado ou Farmácia
- **ProdutoManager**
 - Criação, edição e listagem de produtos
 - Produto pertence a uma empresa
 - Depende de `EmpresaManager`
- **PedidoManager**
 - Criação e edição de pedidos, validações
 - Persistência em `data/pedidos.xml` (Usando `utils/Persistencia.java`)

- Depende de `UsuarioManager` e `EmpresaManager`
- **EntregaManager**
 - Criação de entregas
 - Persistência em `data/entregas.xml` (Usando `utils/Persistencia.java`)
 - Depende de `UsuarioManager`, `EmpresaManager` e `PedidoManager`

Models

- **Usuario**
 - `Cliente`
 - `Dono`
 - `Entregador`
- **Empresa**
 - `Restaurante`
 - `Mercado`
 - `Farmácia`
- `Produto`
- `Pedido`
- `Entrega`

Atributos e getters/setters

Persistência

Persistência dos dados em XML através de `XMLEncoder/Decoder`

- `save(String path, List<T> data)` usando `XMLEncoder`
- `load(String path)` usando `XMLDecoder`

Os Managers usam essa classe para manter a persistência

Padrões de Projeto

Facade

O padrão Facade fornece uma interface unificada para um conjunto de classes de um sistema. No lugar de interagir com vários objetos e fluxos internos, ele utiliza uma única "porta de entrada" que coordena chamadas

No projeto, esse padrão resolve a necessidade de fornecer uma API simples para os testes do EasyAccept.

A oportunidade do uso surge ao perceber que sem isso, seria necessário o "cliente" externo conhecer todos os managers, instanciá-los na ordem correta e gerenciar as dependências manualmente.

A classe Facade constrói os cinco managers no construtor com as dependências corretas e expõe métodos que delegam para o manager adequado: operações de usuário vão para `UsuarioManager`, de empresa para `EmpresaManager`, de produto para `ProdutoManager`, de pedido para `PedidoManager` e de entrega para `EntregaManager`

```
public void criarUsuario(String nome, String email, String senha, String endereco)
    throws Exception {
    usuarioManager.criarUsuario(nome, email, senha, endereco);
}
```

No exemplo acima mostra que ao "criarUsuario" é chamado a Facade delega o trabalho ao `UsuarioManager`. Não existe necessidade dos testes saberem como funciona o sistema internamente.

```
public void entregar(String idEntrega) throws Exception {
    entregaManager.entregar(idEntrega);
}
```

No exemplo acima mostra que ao "entregar" é chamado a Facade delega o trabalho ao `EntregaManager`.